Profiling to Personalisation: Predicting Customer Actions and Spend for Accurate Data Push

Jingyi Zhao, Leia Whitaker, Elena Llinares Perales, Unico Yin

***Abstract*—In this project, the goal was to determine the activity and preferences of customers.**

As a group, we looked at a different aspect of these questions so that we would get an overall view of what behaviour customers have and use these findings to suggest improvements to a local business looking to streamline sales and optimise targeted promotions.

Each group member applied one of the following machine learning algorithms: Random Forest, linear regression, or XGBoost.

After each member had conducted their analysis, the results showed that it was possible to predict the behaviours in question (the accuracy of all models was above 0.7).

I. Introduction

In today's competitive retail landscape, data-driven marketing is essential for understanding individual preferences and predicting behaviour[1],[2]. The proliferation of digital tracking provides rich datasets on demographics, purchases, and campaign interactions. Machine learning on such data uncovers actionable insights for more targeted and efficient strategies[3],[4].

This study leverages the *Customer Personality Analysis* dataset—containing demographic and behavioural features—to model key outcomes: promotion acceptance, channel choice (online vs. offline), customer income, and category-specific spend. By applying and comparing predictive algorithms, we aim to enable personalised decisions that boost engagement, conversion, and overall profitability.

II. Related Works

Machine learning has become a cornerstone for customer analytics, with Random Forest and Decision Tree models frequently delivering high accuracy and clear feature-importance insights in churn and promotional response prediction [5], [6]. Complementary research has examined shopping channel choices (online vs. offline) using ensemble methods to capture complex behavior patterns [7], as well as category-specific spending models for products like wine, which identify key spending drivers [8]. Work on the *Customer Personality Analysis* dataset underscores the importance of demographic variables (income, marital status) and historical campaign interactions as strong predictors of response [9]. Building on these foundations, our study integrates data cleaning, feature engineering (including purchase frequency and total spend), and balanced classification techniques to further probe and enhance the predictability of customer response behaviors across multiple marketing tasks.

III. Methodology

A. Data Source

The study employs the publicly available Customer Personality Analysis dataset (Kaggle, 2021), which merges demographic, behavioural and historical spend attributes for 2240 retail customers.

B. Cleaning

Rows with missing or invalid income (24 missing, 1 set to 666666) and fictitious marital statuses (4 with "YOLO" or "Absurd") were removed. "Alone" was merged into "Single" to reduce sparsity, leaving 2209 customers. Leia filled remaining missing income with the median.

C. Feature Engineering

- Global:
  - **Age** = 2021 − *Year_Birth*
  - **DaysSinceSignup** from *Dt_Customer*
  - **TotalSpent** (sum of five non-wine categories)
  - One-hot encoding for *Education* & *Marital_Status*
- Per-task additions
  - *Elena* creates **PurchaseTypeNum** (web-heavy = 0, store-heavy = 1).
  - *Leia* derives **TotalPurchases**, **Kids** (*Kidhome* + *Teenhome*) and a binned **Age_Group**.
  - *Jingyi* builds **TotalMnt**, **AvgSpendPerPurchase** after Winsorising long-tailed spend columns.

D. Evaluation

- Jennie – promotion response, Random-Forest + SMOTE, F1.
- Elena – next purchase channel, Random-Forest + balance.
- Leia – annual income, LASSO regression, MAE.
- Unico – red-wine spend, Random-Forest regression, R^2.

Classification tasks report Accuracy, Precision, Recall, F1 and ROC-AUC (Jennie 0.75 overall accuracy after tuning; Elena AUC = 0.87). Regression tasks report R^2 and RMSE (Unico $R^2 = 0.71$; Leia $R^2 = 0.75$ on unseen test set). Cross-validated means are provided for robustness; Unico's 5-fold CV yields 0.692 ± 0.038.

IV. Leia

A. Aim

Predicting a customer's annual income based on their attributes from the selected customer personality analysis dataset. The predictive model used in linear regression.

B. Data Preparation

The raw dataset was first preprocessed to ensure data quality, suitability for predictive modelling and consistency across features.

C. Data Cleaning

- The 'year_Birth' feature was converted into an age feature. Unrealistic ages (ages over 120 years old) were removed as likely data errors.
- In the 'Marital_Status' feature, an unusual category labelled 'YOLO' was also removed to avoid ambiguity.
- Missing values in the raw income column were imputed using the dataset's median income. This approach was chosen because income was highly skewed (as shown in Figure 1), and the median is less sensitive to outliers than the mean.
- The Income variable, our target variable, was log-transformed to reduce its highly positive skew. This helps to normalise the distribution and make its relationship to other features more linear, which improves the performance of a linear regression model.

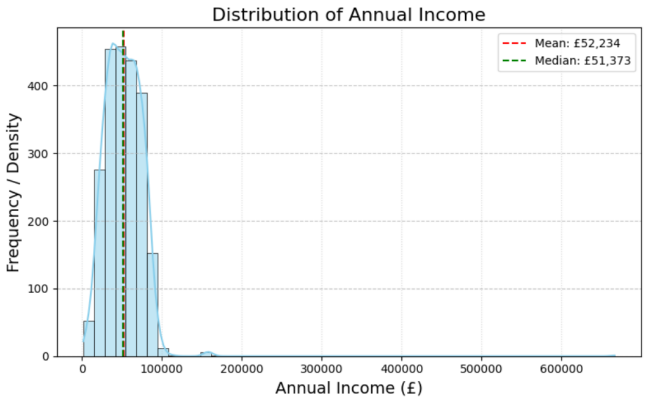


Fig. 1. Histogram illustrating the income distribution across the population

D. Data Splitting

The cleaned dataset was split into training (70%), validation (15%), and test (15%) sets. This helps ensure that model evaluation is unbiased and that performance metrics reflect generalisation to unseen data. The validation set was used for hyperparameter tuning, while the test set was used for the evaluation at the end. As this is a regression task with a continuous target variable, stratified splitting was not applied, as it is primarily suited to classification problems with categorical outcomes.

The subsequent calculated features required for data preparation, such as converting 'Dt_Customer' to 'Customer_Tenure_Days', are only performed on the training set. This is then applied consistently to the validation and test sets to prevent data leakage for unbiased evaluation.

E. Handling Outliers

Box plots were used to identify the outliers in the numerical features. For features with a strong positive skew, a log-transformation was applied to reduce the skew and lessen the impact of extreme values. For the other features, the IQR method was used to cap the outliers; values 1.5 times the interquartile range above or below the quartiles were limited to the upper or lower bounds.

F. Feature Engineering

Several new features were created to potentially capture more predictive information:

- 'Total_Mnt': Total spending across all product categories.
- 'Total_Purchases': Total number of purchases made across all channels.
- 'Kids': Total number of children ('Kidhome' + 'Teenhome').
- 'Age_Group': Continuous Age was binned into categorical groups (young, adult, middle-aged and senior) to account for non-linear patterns in how age might impact income.

G. Categorical Feature Encoding

Nominal categorical features were converted into binary columns using one-hot encoding so that the model could be applied to them.

H. Numerical Feature Scaling

All continuous numerical features were scaled to have a mean of 0 and a standard deviation of 1. This is important for Linear Regression because it is sensitive to the scale of features, where larger numerical ranges can disproportionately influence the model.

I. Applying Linear Regression Model

Linear regression was chosen because the target variable, income, is continuous. It provides strong interpretability; each coefficient represents the expected change in log-transformed income for a one-unit change in a feature, holding all other features constant. This makes it easier to understand which factors influence income and to what extent.

J. LASSO Feature Selection

LASSO regression was applied to enhance interpretability, limit overfitting, and automatically select key features by shrinking less important coefficients to zero. This focuses the model on the most relevant features and reduces noise. It works by exploring a range of alpha values, a

parameter that controls how much the model penalises larger coefficients. Figure 2 depicts how alpha influences

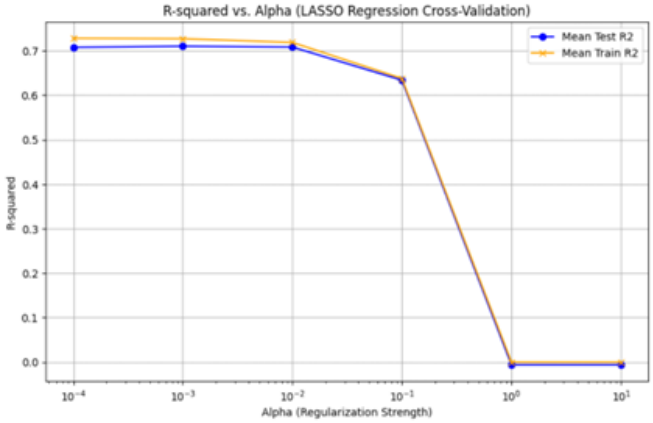


Fig. 2. R-squared Value Across Different Alpha Coefficients in LASSO Regression

the model's R-squared value. The mean test R-squared values closely matched the mean train R-squared values. This suggests that the model fits the data well and is likely to perform reliably on new, unseen data without overfitting. As alpha increases, the R-squared values drop significantly, suggesting underfitting. An alpha value of 0.001 was selected as best, which resulted in 25 features having non-zero coefficients in the final model.

K. Final Results

TABLE I
A TABLE TO SHOW THE MODEL'S RESULTS

Metric	Validation Set	Test Set
MAE	0.1461	0.1669
MSE	0.0463	0.0762
RMSE	0.2152	0.2760
R-squared	0.7941	0.7475

An analysis of the model's absolute magnitude coefficients, an indicator of a feature's influence on the scaled model, revealed that the amount spent on wine products was the most influential factor positively correlated with income. Other significant features included the number of monthly web visits, basic education level, the total number of children, and the number of purchases made with deals. These findings align logically with expectations.

V. Discussion

A. Strengths

The model achieved an R-squared of about 0.75 on the unseen test set. Therefore, it explains approximately 75% of the variance in the log-transformed annual income. This indicates strong predictive performance for a real-world financial variable.

The Mean Absolute Error (MAE) of about 0.167 and Root Mean Squared Error (RMSE) of 0.276 show that the model's predictions are, on average, very close to the actual log-transformed income values, illustrating strong precision.

Furthermore, the model demonstrated strong generalisation as its test set results closely match the validation set results. This indicates that the model is not overfitting and can reliably predict income for new customers.

B. Limitations

Although LASSO helped improve the model, linear regression still assumes a linear relationship between the features and income. Even with transformations, some complex, non-linear patterns may remain unaccounted for, which could explain some of the prediction errors. Furthermore, an R-square of around 0.75 means that about 25% of the income variation is still unexplained. This indicates that there are likely other important factors affecting income that are not captured in the current dataset. The model might, therefore, give biased or incomplete predictions. Lastly, since the model is trained on a specific customer dataset, its performance may not generalise well to other populations.

C. Future Work

Several approaches could be explored in the future to improve the model's predictive power. For example, investigating more complex non-linear models may increase the R-squared by better capturing complex patterns in the data, although this might reduce interpretability. Alternative feature selection methods could also help identify a smaller, more relevant set of features, potentially improving accuracy. Finally, additional feature engineering could create new features with stronger predictive power.

VI. Elena

A. Aim

This study used the Customer Personality Analysis dataset to predict whether customers prefer to buy products in-store or online.

B. Data Preparation

To prepare the data for training, some features were adjusted:

- The year of birth was changed to age because this dataset was created in 2021, and age was an easier parameter to test, as a numerical parameter.
- Total spending on products was also added, which was the sum of all the amounts spent on recorded products (Wine, Gold, Fish, Meat, Fruit, and Sweet)
- Income had null values, and these were adjusted using Median Imputation (replacing null with the median of the training data)
- Purchase Type Num was a feature added to record whether a customer made more than half of their purchases in store (1) or online (0)

Some features were removed from the dataset:

- ID – identifier for each customer
- Year of Birth – Replaced with Age
- Date customer enrolled – the data set was from 2021, not what day or month, and so it was unreliable if changed into years spent enrolled
- Number of web purchases – this was being predicted
- Number of store purchases – this was being predicted
- Promotion accepted 1-5 – not of interest
- Z cost contact – irrelevant
- Z revenue – irrelevant

C. Preprocessing

After removing and adjusting some of the variables, the final step was to encode the categorical variables of Marital Status and Education into binary columns using get_dummies(), a one-hot encoder.

D. Data Splitting

Finally, the data was split into training, validation, and test data (80:10:10, respectively). Including a portion of the data for testing aimed to decrease the chances of overfitting.

E. Selection of a model

To find which model would perform best, all of the different models were tested and rated based on a custom score, which used the standard deviation and the mean of accuracy, precision, and recall to favour a model that had a high value of accuracy, precision, and recall but a small range where these values lie within. [Table II]

TABLE II
MODEL COMPARISON RESULTS

Model	Acc.	Prec.	Rec.	Score
Random Forest	0.8213	0.8267	0.9417	0.8077
Logistic Reg.	0.7602	0.7766	0.9223	0.7468
Decision Tree	0.7285	0.8000	0.8155	0.7434
Support Vector Machine	0.7579	0.7672	0.9385	0.7382

The results showed that the Random Forest model achieved the highest balanced score among the models tested. This was followed by the Decision Tree, with the Support Vector Machine and Logistic Regression performing less well by comparison. Based on these results, the Random Forest classifier was selected for further development.

Random forest is a machine learning method for classification and regression. It trains by using multiple decision trees. In this use case, the forest's outcome is the purchase channel most predicted by the decision trees.

Additionally, the Random Forest model is best suited for classification tasks. This aligns with the study's objective of classifying customers according to their preferred purchasing channel, either online or in-store.

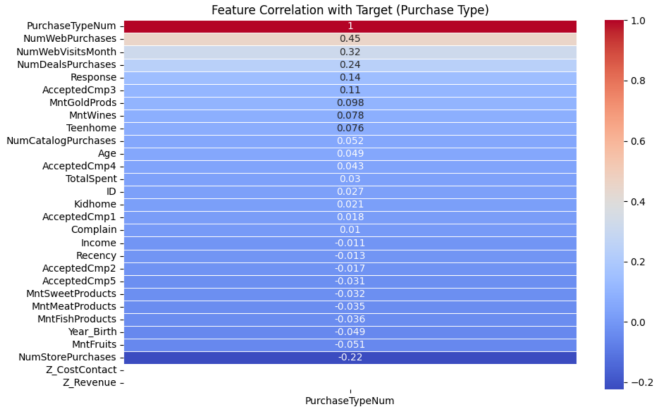


Fig. 3. Feature heat map with correlations to predictive feature

F. Feature Selection

First, a heat map [Figure 3] was created to show the correlation between many of the variables and the Purchase Type. It shows a range of significant values enabling analysis.

Backwards selection was conducted using recursive feature elimination with cross-validation (RFECV). This approach ensured that only the most predictive features were retained, improving interpretability and reducing overfitting risk. Given the large number of demographic, behavioural, and promotional features in the dataset, this method allowed identification of a minimal feature set that preserved model performance. This reduction in features is also helpful to discard the redundant or weakly predictive ones due to categorical data encoding. Additionally, the use of cross-validation in RFECV ensured that feature elimination decisions were informed by generalisation performance, not just training accuracy.

RFECV allowed a score to favour some features over others. Since categorisation was the aim, the same scorer that was created for model selection was used for feature selection so that accuracy, recall, and precision were optimised to be balanced and within a small range.

The features that were chosen include:

1) Age
2) Income
3) Total spending
4) Amount spent on wine
5) Amount spent on meat
6) Amount spent on fish
7) Amount spent on sweet products
8) Amount spent on gold products
9) Days since last shop (Recency)
10) Number of visits to the website in the last month

G. Hyperparameter Tuning

The number of trees in the random forest was the only hyperparameter that was tuned. Since the model categorised all the data into two classes, when optimising

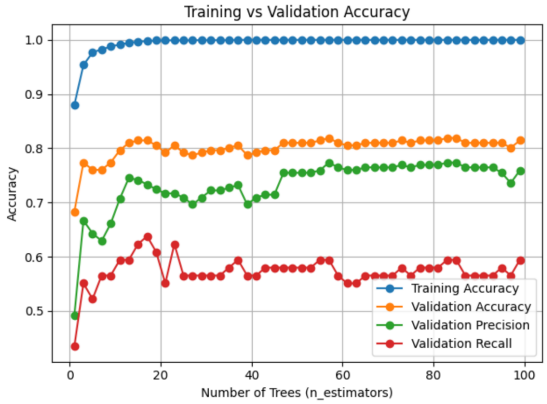


Fig. 4. Training Accuracy vs Validation Accuracy, Precision and Recall Across Different N Estimators

precision, recall, and accuracy, all were equally important. Therefore, all three metrics of the validation data were plotted to choose a number of trees that would balance them. [Figure 4]

Recall (0.64) and accuracy (0.81) were highest at n_estimators=17 [Figure 4], where precision (0.73) was almost at its maximum. As can be seen below (Table III), this balance was further seen with the confusion matrix of the validation data [Figure 5], where both the False Positives (16) and False Negatives (25) were kept relatively low.

TABLE III
Training and Validation Results

	Accuracy	Precision	Recall
Training	0.9977349943	1.0	0.9924953096
Validation	0.8144796380	0.733333333	0.6376811594

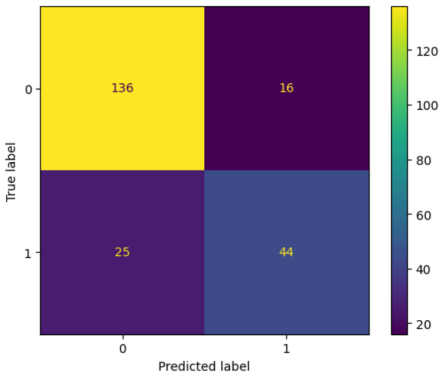


Fig. 5. Validation Data Confusion Matrix

H. Discussion

The final test of the model's performance was done with the test dataset set aside from the beginning. These test values are very similar to the validation data, which shows that the model does not suffer from overfitting. The similar confusion matrices [Figure 6] further reinforce this.

TABLE IV
Testing Results

	Accuracy	Precision	Recall
Training	0.9977349943	1.0	0.9924953096
Validation	0.8144796380	0.733333333	0.6376811594
Test	0.8371040724	0.7413793103	0.671875

One limitation might be that the model uses 10 features to predict the type of purchase. This could possibly cause overfitting because it takes into account so many variables, reducing the degrees of freedom.

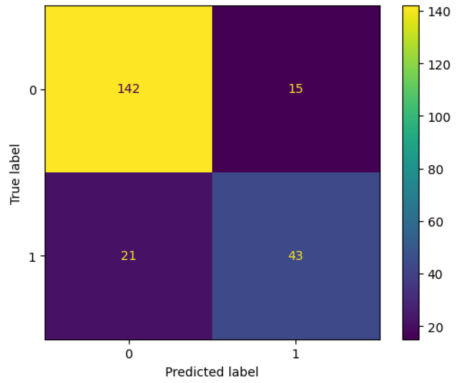


Fig. 6. Testing Data Confusion Matrix

When looking at the confusion matrix, it can easily be seen that in this dataset, a larger portion of people buy products online than in-store (around 155 online and 65 in-store). This will likely skew the data if this model is presented with a dataset where the split is more even or even skewed towards in-store purchases.

Overall, the random forest model creates well-balanced predictions of customer preferences for in-store or online purchases, with a test accuracy of 0.84, which could, for example, be used to better aim online promotions and in-store discounts depending on where customers shop.

VII. Product Consumption - Unico Yin

A. Aim and Problem Framing

This study predicts each active customer's two-year red-wine expenditure (MntWines) using features in the dataset and behavioural data. Accurate spend forecasting enables marketing teams to filter prospects based on economic value, adjust discount depth and allocate premium inventory, replacing one-size-fits-all offers with profit-driven, customer-level targeting.

B. Data Overview

The working sample comprises 2 209 customers extracted from the original 2 240-row dataset after removing (i) 24 records lacking income, (ii) one implausible income

outliers (< 0 or > 600000) and (iii) the fictitious marital labels YOLO and Absurd; the marital status Alone was merged into Single. (iv) removed three records with age greater than 120 years old. Each record supplies demographics (Year_Birth, Education, Marital_Status, Income), household composition (Kidhome, Teenhome), tenure (Dt_Customer), multichannel behaviour and six product-level spending figures. Two engineered variables—Age and DaysSinceSignup—and an aggregated TotalSpent (non-wine categories) expand the usable feature pool to 27 columns. These predictors span economic capacity, engagement intensity and temporal loyalty, giving the model a balanced view of factors that can plausibly drive wine purchasing.

C. Feature Engineering and Selection

Two numerical features were engineered to enrich behavioural context:

Age — calculated as the birth year in 2021, which is the same as when the dataset in Kaggle was created. Used to predict what age people are more likely to buy red wine.

DaysSinceSignup – days elapsed since registration (Dt_Customer), reflecting tenure-based loyalty.

Categorical fields Education and Marital_Status were one-hot encoded with the first level dropped, yielding 15 dummy variables. A consolidated TotalSpent variable (sum of MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds) represents overall purchasing power outside the target category.

A preliminary Random-Forest model trained on the full 27-feature matrix produced impurity-based importance scores. Variables with importance ¿ 0.01 were retained, leaving nine predictors that together explained 94 % of total importance (Table 1). This pruning reduces noise and simplifies deployment while slightly improving hold-out performance.

TABLE V
TOP-9 RETAINED PREDICTORS (IMPORTANCE > 0.01)

Rank	Feature	Importance
1	Income	**0.544**
2	NumStorePurchases	0.107
3	TotalSpent	0.095
4	NumWebPurchases	0.091
5	Recency	0.054
6	Age	0.053
7	NumDealsPurchases	0.023
8	Education_PhD	0.020
9	Education_Graduation	0.013

D. Modelling Setup

A Random-Forest Regressor was chosen for its ability to capture non-linear interactions, handle mixed-scale predictors and provide intrinsic feature-importance scores. All experiments share the following protocol:

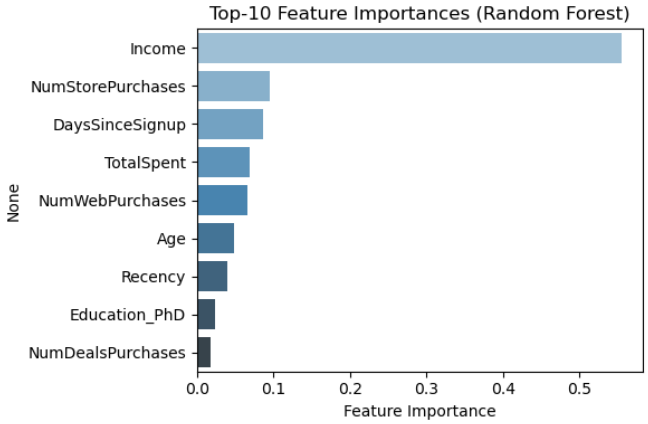


Fig. 7. Feature Importance

Train–test split The cleaned dataset (n = 2 209) is randomly split 80

Hyper-parameters Initial screening showed little gain beyond 500 trees, so the final model uses n_estimators = 500, max_depth=None, min_samples_split = 2, min_samples_leaf = 1, bootstrap=True.

Feature variants Two matrices are evaluated: – Full 27 predictors after preprocessing. – Reduced 9 predictors retained via the ¿ 0.01 importance threshold (Section 3).

Cross-validation Within the training fold, 5-fold cross-validation estimates generalisation error; the same folds are reused for both feature variants to guarantee comparability.

Software stack All models are implemented in scikit-learn and executed on an AMD-9800X3D-based PC; total training time < 3s per run.

This set-up balances computational efficiency with robust error estimation and serves as the basis for the performance comparison in Section 5.

E. Results

1) Hold-out Performance: The final nine-predictor Random-Forest model reaches an R^2 of 0.710 on the 20 % hold-out set, explaining 71 % of the variance in two-year red-wine spending. The corresponding mean-squared error is 42 501, indicating an average prediction deviation of roughly ±206 (root-MSE) per customer.

2) Cross-validation: Five-fold cross-validation on the training data yields

TABLE VI
FIVE–FOLD CROSS–VALIDATION R^2 SCORES

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean ± SD
0.754	0.680	0.700	0.661	0.664	$\mathbf{0.692 \pm 0.038}$

The tight spread confirms that the model's accuracy is stable across different subsets and not driven by a fortunate split.

3) Visual Diagnostics: Figure 8 plots predicted versus actual spending; points cluster tightly around the 45° line up to around 1400, with only a few high-spend outliers slightly under-estimated. Figure 9 shows a residual histogram centred on zero and approximately symmetric, indicating no visible heteroscedasticity or systematic bias. These graphics reinforce the quantitative metrics and suggest the model generalises well enough for deployment in a live personal-targeting pipeline.

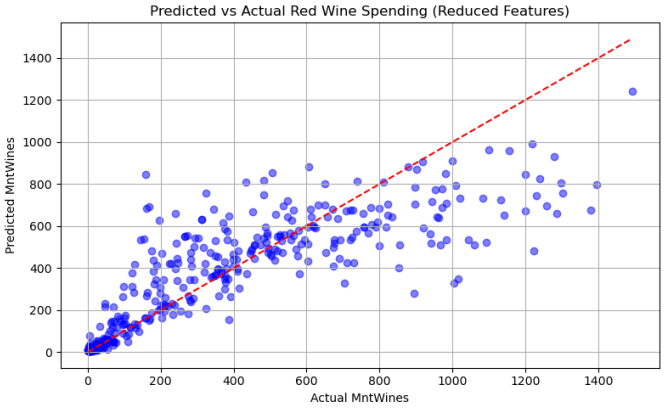


Fig. 8. Predicted vs. actual red-wine spending.

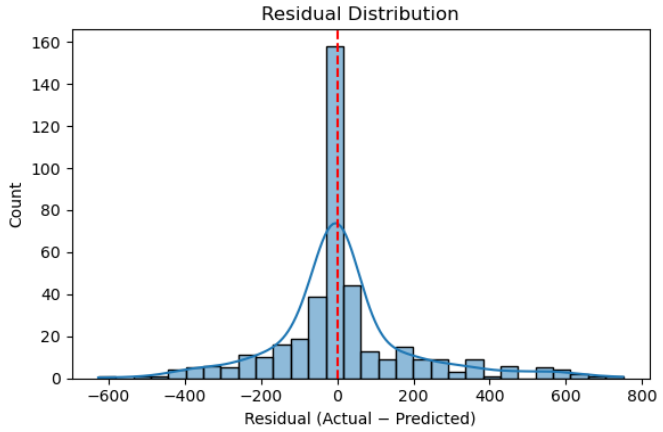


Fig. 9. Residual Histogram between Actual and Predicted Consumption

F. Discussion

The nine-predictor Random-Forest model delivers two insights of direct managerial value. First, economic capacity dominates purchase behaviour. Income alone explains more than half of the model's decision variance, confirming that discretionary spending power remains the primary lever for premium categories. Yet income is not sufficient: NumStorePurchases and TotalSpent jointly account for another 20 %, indicating that active, multichannel shoppers convert their budgets into tangible basket sizes. Second, engagement recency outweighs pure tenure. While DaysSinceSignup contributes only 1 %, the short-term activity proxy Recency carries five-times more importance (5%), suggesting that fresh interactions, rather than raw customer age, signal readiness to buy wine.

From an operational perspective the compact model offers three advantages. Parsimony—nine easily captured fields suffice, reducing storage and ETL overhead. Interpretability—tree-based importance ranks are intuitive for non-technical marketers, facilitating buy-in. Scalability—prediction latency is ¡5 ms per user on a consumer laptop, enabling real-time personalisation in e-mail or on-site banners.

Nevertheless, two caveats merit attention. The model slightly under-predicts the extreme right tail (> 1400), implying that high-spend connoisseurs exhibit qualitative drivers (e.g., vintage preference, brand prestige) absent from the dataset. In addition, temporal aggregation across two years masks seasonality: holiday spikes or harvest-driven promotions are flattened. Both issues could be mitigated by adding product-level metadata and splitting the time axis into quarterly windows.

From a marketing standpoint, these findings translate into an actionable value-response matrix: high-income, high-engagement users warrant premium offers (e.g., limited vintages), whereas mid-income but highly engaged customers may respond better to loyalty-tier discounts that raise overall basket size.

Overall, the model substantiates that a lean profile can forecast 70% of expenditure variance, giving the marketing team a quantitative backbone for tiered discounting, premium-club invitations and inventory planning, while leaving room for future enhancements through richer behavioural signals.

V. Promotion Response – Jingyi Zhao

A. Aim

This study aims to build a Random Forest classification model to predict whether a customer will respond to a promotion based on their past campaign engagement, discount-based purchases, and demographic features. The goal is to help businesses identify high-potential responders and better target future marketing efforts.

B. Model Selection

A Random-Forest Regressor was chosen because the data's structure matches the algorithm's comparative advantages. The predictors mix binary campaign-acceptance flags with a count variable (`NumDealsPurchases`), giving a categorical–numerical blend that tree ensembles handle *natively* without linearity assumptions. The target, `TotalMnt`, is highly skewed and contains outliers, conditions under which Random Forest remains stable. Moreover, its built-in feature-importance scores allow empirical ranking of

promotion behaviors—such as specific `AcceptedCmp` indicators and discount-purchase frequency—by their impact on lifetime customer value.

C. Filtering Data

Before training the model, we applied Winsorization and standardization to two key numerical variables: TotalMnt (total customer spending) and NumDealsPurchases (number of discount-based purchases). Since TotalMnt exhibits a long-tailed distribution, with some customers spending far above typical levels, using the raw data could cause the model to overfit to extreme cases. To address this, we clipped the values between the 1st and 99th percentiles, effectively reducing the influence of outliers while preserving the overall structure of the data.

TABLE VII
DESCRIPTIVE STATISTICS OF `TotalMnt` AND `NumDealsPurchases` AFTER STANDARDIZATION AND WINSORIZATION

Statistic	`TotalMnt`	`NumDealsPurchases`
count	2.208000e+03	2.208000e+03
mean	4.344351e-17	1.818191e-16
std	1.000227e+00	1.000227e+00
min	-9.856606e-01	-1.282403e+00
25%	-8.940809e-01	-7.240657e-01
50% (median)	-3.530808e-01	-1.657258e-01
75%	7.114398e-01	3.926086e-01
max	2.598177e+00	4.261885e+00

D. Features Engineering

Based on the cleaned dataset, several important features were engineered to enhance the model's ability to capture patterns in customer behavior and promotional response. `TotalMnt` was created by aggregating spending across multiple product categories, representing each customer's total expenditure. `TotalPurchases` and `AvgSpendPerPurchase` were derived to reflect purchase frequency and spending intensity.

Categorical variables such as `Education` and `Marital_Status` were one-hot encoded to allow tree-based models to interpret them without assuming ordinal relationships. Key numerical variables, including `Income`, `Recency`, and `Tenure`, were standardized using Z-score normalization to ensure consistent scaling across features.

These transformations converted raw inputs into a more structured and meaningful format, providing a solid foundation for predictive modeling and improving both accuracy and interpretability.

E. Train & Test Split

After outlier treatment and standardization, we compared three data-split schemes: 60/20/20, 70/15/15, and 80/10/10 (train/val/test). The 70/15/15 split proved optimal, providing stable validation precision and minimal gap between training and validation, indicating strong generalization. The 80/10/10 split yielded noisy validation results due to its small validation set, while 60/20/20 underfit from limited training data. Moreover, the 70/15/15 configuration maintained consistent performance over various `min_impurity_decrease` values, confirming its suitability.

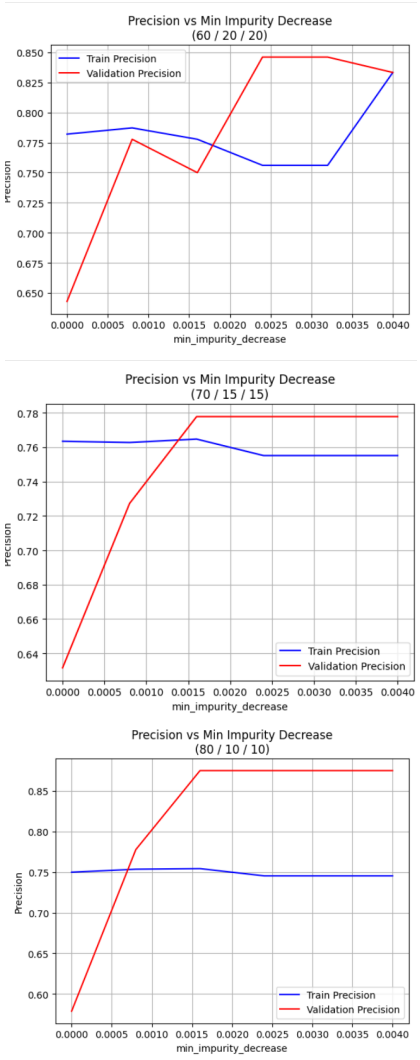


Figure 6. Precision vs. `min_impurity_decrease` for different train-validation-test splits

F. Random Forest

TABLE VIII
PERFORMANCE OF RANDOM FOREST BEFORE BALANCING

Class	Precision	Recall	F1-score	Support
0 (non-responders)	0.88	0.98	0.93	286
1 (responders)	0.63	0.24	0.35	50
Accuracy	–	–	0.866	336

In the first round of Random Forest testing, the model achieved an overall accuracy of 86.6%. For the majority class (*non-responders*), performance was strong, with precision of 88% and recall of 96%, indicating the model could effectively identify customers unlikely to respond.

However, for the minority class (*responders*), the recall was only 12%, meaning most actual responders were missed, despite a decent precision of 58%. This imbalance stems from the skewed class distribution, where responders make up a small portion of the data, leading the model to favor predicting the majority class.

To improve minority class detection, we applied SMOTE oversampling and class weight adjustment in later stages, aiming to boost recall and achieve better class balance.

TABLE IX
CLASSIFICATION REPORT AFTER CLASS WEIGHT ADJUSTMENT: {0: 1, 1: 4}

Class	Precision	Recall	F1-score	Support
0 (non-responders)	0.89	0.90	0.89	286
1 (responders)	0.38	0.36	0.37	50
Accuracy			0.82	336
Macro avg	0.64	0.63	0.63	336
Weighted avg	0.81	0.82	0.82	336

Before sample-level balancing, we set `class_weight` to emphasize the minority class, but the recall for class 1 (responders) was only 0.36. This highlights that class imbalance still led the model to favor the majority class, limiting its ability to detect true responders.

TABLE X
CLASSIFICATION REPORT AFTER SMOTE OVERSAMPLING

Class	Precision	Recall	F1-score	Support
0 (non-responders)	0.64	0.94	0.76	496
1 (responders)	0.89	0.49	0.63	512
Accuracy			0.71	1008
Macro avg	0.76	0.71	0.70	1008
Weighted avg	0.77	0.71	0.69	1008

After applying SMOTE for sample-level oversampling, the model's ability to identify class 1 (*responders*) improved significantly, recall increasing to 0.49 and precision reaching 0.89. This indicates significant progress in recognizing the positive class and a better balance between classes overall. However, since hyperparameter tuning (e.g., `GridSearch`) had not yet been applied at this stage, there was still room to improve the generalization performance of the model.

G. GridSearchCV

We then used `GridSearchCV` to fine-tune the hyperparameters of the Random Forest model and improve both performance and generalization. The parameter search covered `n_estimators` (number of trees: [100, 300, 500]), `max_depth` ([None, 5, 10, 20]), `min_samples_split` ([2, 5, 10]), and `max_features` (['sqrt', 0.3, 0.5]). The following plots illustrate how different parameter combinations affected model performance and helped identify the optimal setup.

TABLE XI
CLASSIFICATION REPORT AFTER GRIDSEARCHCV TUNING: {MAX_DEPTH: NONE, MAX_FEATURES: 0.5, MIN_SAMPLES_SPLIT: 10, N_ESTIMATORS: 300}

Class	Precision	Recall	F1-score	Support
0 (non-responders)	0.70	0.87	0.78	565
1 (responders)	0.83	0.62	0.71	565
Accuracy			0.75	1130
Macro avg	0.76	0.75	0.74	1130
Weighted avg	0.76	0.75	0.74	1130

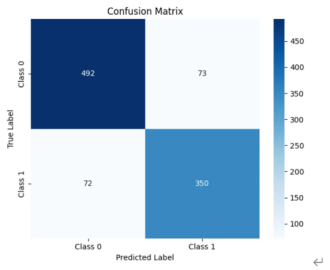


Fig. 10. Confusion matrix of the final Random Forest model after `GridSearchCV` tuning

After tuning, the model's precision, recall, and F1 for class 1 (responders) rose to 0.83, 0.62, and 0.71—substantially above initial levels. Overall accuracy reached 75%, with macro and weighted averages around 0.75. These gains show that adjusting `n_estimators`, `max_features`, and `min_samples_split` effectively improved minority detection while preserving general performance and generalization.

H. XGBoost

Despite the improvements, it is sought to have even better accuracy, and `XGBoost` is adopted. The code is written and trained for the `XGBoost` model, and the results are shown below:

TABLE XII
CLASSIFICATION REPORT OF THE XGBOOST MODEL

Class	Precision	Recall	F1-score	Support
0 (non-responders)	0.70	0.87	0.77	565
1 (responders)	0.83	0.62	0.71	565
Accuracy			0.75	1130
Macro avg	0.76	0.75	0.74	1130
Weighted avg	0.76	0.75	0.74	1130

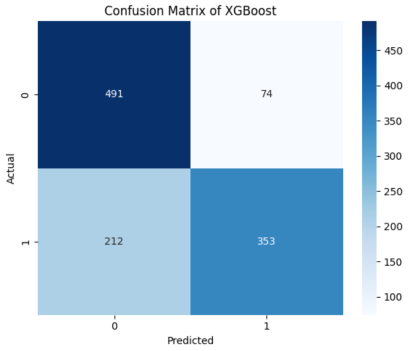


Fig. 11. Confusion matrix of the final Random Forest model after `GridSearchCV` tuning

The XGBoost model achieved solid accuracy 75%, with balanced performance across both classes. It showed strong results for the majority class (class 0), with 87% recall and 70% precision, and performed well on the minority class (class 1), achieving 83% precision and 62% recall. Overall, the model handles class imbalance effectively and enhances responder detection, making it well-suited for customer-targeting tasks.

I. Discussion

To further improve model interpretability, we used `feature_importances_` to analyze the importance of each variable from both global and local perspectives. The resulting chart highlights that `TotalMnt` (total spending), `AcceptedCmp1`, and `AcceptedCmp5` are the most influential predictors. These results suggest that customers' overall expenditure and prior campaign acceptance are key drivers of promotional responsiveness.

This analysis offers valuable insights for refining the model by focusing on high-impact variables, which may lead to improvements in both predictive accuracy and explanatory power.

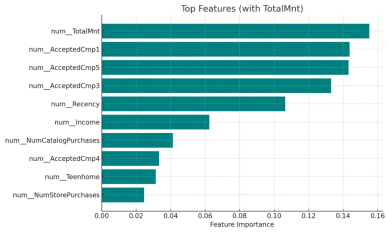


Fig. 12. Confusion matrix of the final Random Forest model after `GridSearchCV` tuning

While the model performs well, there is still room for improvement. Some misclassifications suggest that current features may not fully capture customer behavior, and the limited dataset may affect generalization. Future work could include incorporating more behavioral data (e.g., web activity) and exploring advanced models such as ensemble methods or deep learning approaches.

In summary, the promotion response prediction model demonstrates strong accuracy, effective minority class detection, and clear business relevance. It provides a solid foundation for future precision marketing strategies.

VIII. Conclusion

The predictive analysis project demonstrated the ability for actionable business insights to be formed from a Customer Analysis dataset. Through data cleaning, targeted feature engineering, and appropriate model selection, the team built strong models addressing different business objectives:

- Jennie's Random Forest model predicted promotion response with 0.75 accuracy.
- Elena's Random Forest model identified purchase channel preferences (accuracy: 0.84).
- Leia's Linear Regression with LASSO explained 75% of income variance, providing interpretable financial information.
- Unico's Random Forest Regressor predicted red-wine spending with an R^2 of 0.71.

A. Limitations

Despite solid metrics, the models capture only slices of human behaviour; no algorithm can fully reflect its complexity. They are valuable for insight, not absolute truth. In addition, data-collection bias limits external validity: although cross-validation shows internal robustness, models trained on this single customer cohort may perform differently in other markets.

B. Opportunities for Future Improvement

Finally, further elements could be explored to address the limitations for future work:

- The data set could incorporate additional data sources to provide a more complete picture of customer behaviour and potentially reduce unexplained variance.
- Our research focused on forming static predictions. Future work could develop models that account for temporal dynamics to form Dynamic Prediction.
- For the classification models (like promotion response or purchase channel), integrating cost-benefit analysis could improve analysis by aligning predictions with business value.

IX. References

References

[1] T. H. Davenport and J. G. Harris, *Competing on Analytics: The New Science of Winning*. Boston, MA: Harvard Business Press, 2007.

[2] M. Wedel and P. K. Kannan, "Marketing analytics for data-rich environments," *Journal of Marketing*, vol. 80, no. 6, pp. 97–121, 2016.

[3] A. Imam, "Customer Personality Analysis Dataset," Kaggle, 2021. [Online]. Available: https://www.kaggle.com/datasets/imakash3011/customer-personality-analysis

[4] S. Chatterjee, N. P. Rana, K. Tamilmani, and A. Sharma, "The next generation of intelligent marketing: A review and research agenda," *International Journal of Information Management*, vol. 56, p. 102289, 2021.
[5] W. Verbeke, D. Martens, C. Mues, and B. Baesens, "Building comprehensible customer churn prediction models with advanced rule induction techniques," *Expert Systems with Applications*, vol. 38, no. 3, pp. 2354–2364, 2012.
[6] B. Lariviere and D. Van den Poel, "Predicting customer retention and profitability by using random forests and regression forests techniques," *Expert Systems with Applications*, vol. 29, no. 2, pp. 472–484, 2005.
[7] B. Yoo and Y. K. Kim, "Online shopping orientation and online retail store image," *Journal of Retailing and Consumer Services*, vol. 19, no. 1, pp. 20–28, 2012.
[8] J. Dawes, "Regularities in buyer behaviour and brand performance: The case of wine," *Australasian Marketing Journal*, vol. 16, no. 1, pp. 42–55, 2008.
[9] A. Imam, "Customer Personality Analysis Dataset," Kaggle, 2021. [Online]. Available: https://www.kaggle.com/datasets/imakash3011/customer-personality-analysis
[10] D. Cox, "Predicting consumption, wine involvement and perceived quality of Australian red wine," *Journal of Wine Research*, vol. 20, no. 3, pp. 209–229, 2009. doi: 10.1080/09571260903450963.
[11] K. J. Chang, M. W. L. Thach, and J. Olsen, "Wine and health perceptions: Exploring the impact of gender, age and ethnicity on consumer perceptions of wine and health," *Wine Economics and Policy*, vol. 5, no. 2, pp. 105–113, 2016. doi: 10.1016/j.wep.2016.09.001.
[12] M. C. Rodríguez-Donate, M. E. Romero-Rodríguez, and V. J. Cano-Fernández, "Wine consumption preferences among generations X and Y: an analysis of variability," *British Food Journal*, vol. 123, no. 11, pp. 3557–3575, 2021. doi: 10.1108/BFJ-12-2020-1156.
[13] T. Garcia, R. Barrena, and I. Grande, "The wine consumption preferences of young people: a Spanish case study," *International Journal of Wine Business Research*, vol. 25, no. 2, pp. 94–107, 2013. doi: 10.1108/IJWBR-2012-0007.
[14] M. C. Rodríguez-Donate, M. E. Romero-Rodríguez, V. J. Cano-Fernández, and G. Guirao-Pérez, "Gender and wine consumption: sociodemographic profiles," *British Food Journal*, vol. 122, no. 1, pp. 242–257, 2020. doi: 10.1108/BFJ-02-2019-0128.

X. Appendix

A. Appendix: Jennie's Code

Listing 1. Jennie's Python code for promotion response prediction

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

df = pd.read_csv(r'C:\Users\Lenovo\Desktop\
    marketing_campaign_CLEANED.csv', sep=',')

df['TotalMnt'] = df[['MntWines', 'MntFruits', '
    MntMeatProducts',
                    'MntSweetProducts', '
    MntGoldProds']].sum(axis=1)

numeric_features = ['TotalMnt', 'NumDealsPurchases']

for col in numeric_features:
    lower = df[col].quantile(0.01)
    upper = df[col].quantile(0.99)
    df[col] = df[col].clip(lower, upper)

scaler = StandardScaler()
df[numeric_features] = scaler.fit_transform(df[
    numeric_features])
print(df[numeric_features].describe())

df = pd.read_csv("C:/Users/Lenovo/Desktop/Grade1/Data
    ␣Science/A2/marketing_campaign.csv", sep='\t')

df['TotalMnt'] = df[['MntWines', 'MntFruits', '
    MntMeatProducts', 'MntSweetProducts', '
    MntGoldProds']].sum(axis=1)
features = ['AcceptedCmp1', 'AcceptedCmp2', '
    AcceptedCmp3', 'AcceptedCmp4', 'AcceptedCmp5', '
    NumDealsPurchases', 'Response']
df = df[features + ['TotalMnt']].dropna()

for col in ['TotalMnt', 'NumDealsPurchases']:
    lower, upper = df[col].quantile([0.01, 0.99])
    df[col] = df[col].clip(lower, upper)

scaler = StandardScaler()
df[['TotalMnt', 'NumDealsPurchases']] = scaler.
    fit_transform(df[['TotalMnt', 'NumDealsPurchases'
    ]])

X = df[['AcceptedCmp1', 'AcceptedCmp2', 'AcceptedCmp3
    ', 'AcceptedCmp4', 'AcceptedCmp5', '
    NumDealsPurchases']]
y = df['Response']

splits = [(0.6, 0.2, 0.2), (0.7, 0.15, 0.15), (0.8,
    0.1, 0.1)]
split_labels = ['60␣/␣20␣/␣20', '70␣/␣15␣/␣15', '80␣/
    ␣10␣/␣10']

min_impurity_vals = np.linspace(0.0, 0.004, 6)
plt.figure(figsize=(18, 5))

for i, (train_size, val_size, test_size) in enumerate
    (splits):
    train_X, temp_X, train_y, temp_y =
    train_test_split(X, y, train_size=train_size,
    stratify=y, random_state=42)
    val_ratio = val_size / (val_size + test_size)
    val_X, test_X, val_y, test_y = train_test_split(
    temp_X, temp_y, train_size=val_ratio, stratify=
    temp_y, random_state=42)

    train_precisions = []
    val_precisions = []

    for min_impurity in min_impurity_vals:
        rf = RandomForestClassifier(
    min_impurity_decrease=min_impurity, random_state
    =42)
        rf.fit(train_X, train_y)

        train_pred = rf.predict(train_X)
        val_pred = rf.predict(val_X)

        train_precisions.append(precision_score(
    train_y, train_pred, zero_division=0))
        val_precisions.append(precision_score(val_y,
    val_pred, zero_division=0))

    plt.subplot(1, 3, i + 1)
```

```
    plt.plot(min_impurity_vals, train_precisions,
    label='Train Precision', color='blue')
    plt.plot(min_impurity_vals, val_precisions, label
    ='Validation Precision', color='red')
    plt.title(f'Precision vs Min Impurity Decrease\n
    ({split_labels[i]})')
    plt.xlabel('min_impurity_decrease')
    plt.ylabel('Precision')
    plt.legend()
    plt.grid(True)

plt.tight_layout()
plt.show()
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import classification_report
import pandas as pd

param_grid = {
    'class_weight': [{0: 1, 1: w} for w in [1, 2, 3,
    4, 5]]
}

rf = RandomForestClassifier(random_state=42)

grid = GridSearchCV(rf, param_grid, scoring='f1', cv
    =5)
grid.fit(X_train, y_train)

best_rf = grid.best_estimator_
val_preds = best_rf.predict(X_val)

print("Best class_weight:", grid.best_params_)
print(classification_report(y_val, val_preds))
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE
import pandas as pd

df = pd.read_csv("C:/Users/Lenovo/Desktop/Grade1/Data
    Science/A2/marketing_campaign.csv", sep='\t')

df['TotalMnt'] = df[['MntWines', 'MntFruits', '
    MntMeatProducts', 'MntFishProducts',
                     'MntSweetProducts', '
    MntGoldProds']].sum(axis=1)
promotion_cols = ['AcceptedCmp1', 'AcceptedCmp2', '
    AcceptedCmp3',
                  'AcceptedCmp4', 'AcceptedCmp5', '
    Response', 'NumDealsPurchases']
X = df[promotion_cols]
y = df['TotalMnt']

threshold = y.quantile(0.75)
y_binary = (y > threshold).astype(int)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(
    X_scaled, y_binary)

X_train, X_test, y_train, y_test = train_test_split(
    X_resampled, y_resampled, test_size=0.3,
    random_state=42)

rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)

print(classification_report(y_test, y_pred))
```

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split,
     GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
from imblearn.over_sampling import SMOTE

df = pd.read_csv("C:/Users/Lenovo/Desktop/Grade1/Data
    Science/A2/marketing_campaign.csv", sep='\t')
df = df[df['Income'].notna()]  #

selected_features = ['AcceptedCmp1', 'AcceptedCmp2',
    'AcceptedCmp3',
                     'AcceptedCmp4', 'AcceptedCmp5',
    'NumDealsPurchases', 'Response']
df = df[selected_features]

X = df.drop('Response', axis=1)
y = df['Response']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(
    X_scaled, y)

X_train, X_test, y_train, y_test = train_test_split(
    X_resampled, y_resampled, test_size=0.3,
    random_state=42, stratify=y_resampled)

param_grid = {
    'n_estimators': [100, 300, 500],
    'max_depth': [None, 5, 10, 20],
    'min_samples_split': [2, 5, 10],
    'max_features': ['sqrt', 0.3, 0.5]
}

rf = RandomForestClassifier(random_state=42)
grid = GridSearchCV(rf, param_grid, cv=5, scoring='f1
    ', n_jobs=-1)
grid.fit(X_train, y_train)

best_rf = grid.best_estimator_
y_pred = best_rf.predict(X_test)

print("Best Parameters:", grid.best_params_)
```

```
print(classification_report(y_test, y_pred))
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
from xgboost import XGBClassifier
from imblearn.over_sampling import SMOTE

df = pd.read_csv("C:/Users/Lenovo/Desktop/Grade1/Data
    ␣Science/A2/marketing_campaign.csv", sep='\t')
df = df[df['Income'].notna()]

features = ['AcceptedCmp1', 'AcceptedCmp2', '
    AcceptedCmp3',
            'AcceptedCmp4', 'AcceptedCmp5', '
    NumDealsPurchases']
X = df[features]
y = df['Response']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(
    X_scaled, y)

X_train, X_test, y_train, y_test = train_test_split(
    X_resampled, y_resampled, test_size=0.3,
    random_state=42, stratify=y_resampled)

xgb_model = XGBClassifier(use_label_encoder=False,
    eval_metric='logloss', random_state=42)
xgb_model.fit(X_train, y_train)

y_pred = xgb_model.predict(X_test)
print(classification_report(y_test, y_pred))
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
from xgboost import XGBClassifier
from imblearn.over_sampling import SMOTE

df = pd.read_csv("C:/Users/Lenovo/Desktop/Grade1/Data
    ␣Science/A2/marketing_campaign.csv", sep='\t')
df = df[df['Income'].notna()]

features = ['AcceptedCmp1', 'AcceptedCmp2', '
    AcceptedCmp3',
            'AcceptedCmp4', 'AcceptedCmp5', '
    NumDealsPurchases']
X = df[features]
y = df['Response']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(
    X_scaled, y)

X_train, X_test, y_train, y_test = train_test_split(
    X_resampled, y_resampled, test_size=0.3,
    random_state=42, stratify=y_resampled)

xgb_model = XGBClassifier(eval_metric='logloss',
    random_state=42)
xgb_model.fit(X_train, y_train)
y_pred = xgb_model.predict(X_test)

conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="
    Blues", xticklabels=[0, 1], yticklabels=[0, 1])
plt.title("Confusion␣Matrix␣of␣XGBoost")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()
```

```
import time

start = time.time()
xgb_model.fit(X_train, y_train)
end = time.time()

print(f"XGBoost␣training␣time:␣{end␣-␣start:.2f}␣
    seconds")
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from sklearn.metrics import confusion_matrix

cm = np.array([[492, 73],
               [72, 350]])

labels = ['Class␣0', 'Class␣1']

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
    xticklabels=labels, yticklabels=labels)

plt.title('Confusion␣Matrix')
plt.xlabel('Predicted␣Label')
plt.ylabel('True␣Label')
plt.tight_layout()
plt.show()
```

B. Appendix: Unico's Code

Listing 2. Unico's Python code for promotion response prediction

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error,
    r2_score
import matplotlib.pyplot as plt
from sklearn.model_selection import cross_val_score


df = pd.read_csv('marketing_campaign_CLEANED.csv',
    sep=',')
df['Dt_Customer'] = pd.to_datetime(df['Dt_Customer'],
     format='%d-%m-%Y')
ref_date = pd.to_datetime('2021-01-01')
df['DaysSinceSignup'] = (ref_date - df['Dt_Customer'
    ]).dt.days
df['Age'] = 2021 - df['Year_Birth']
df = df[df['Age'] <= 100]
df = df.dropna(subset=['Income'])
df = pd.get_dummies(df, columns=['Education', '
    Marital_Status'], drop_first=True)
df['TotalSpent'] = df[[
    'MntFruits', 'MntMeatProducts', 'MntFishProducts'
    ,
    'MntSweetProducts', 'MntGoldProds'
]].sum(axis=1)
df['HighSpenderFlag'] = (df['TotalSpent'] > 250).
    astype(int)

features = [
    'Age', 'Income', 'Kidhome', 'Teenhome', 'Recency'
    , 'DaysSinceSignup',
    'NumWebPurchases', 'NumStorePurchases', '
    NumDealsPurchases',
    'TotalSpent', 'HighSpenderFlag'
] + [col for col in df.columns if col.startswith('
    Education_') or col.startswith('Marital_Status_')
    ]
target = 'MntWines'
Setting_Amount = 30
df['TotalSpendingAll'] = df[[
    'MntWines', 'MntFruits', 'MntMeatProducts',
    'MntFishProducts', 'MntSweetProducts', '
    MntGoldProds'
]].sum(axis=1)

df_active = df[df['TotalSpendingAll'] >=
    Setting_Amount].copy()
X = df_active[features]
y = df_active[target]
print("Features in X:", X.columns.tolist())      #
                    'ID'    'Dt_Customer'
rf_init = RandomForestRegressor(n_estimators=100,
    random_state=42)
rf_init.fit(X, y)
importances = rf_init.feature_importances_
print("Feature Importances (Initial):", importances)
features_all = X.columns
coef_table = pd.DataFrame({
    "Feature": features_all,
    "Importance": importances
}).sort_values(by="Importance", ascending=False)
important_features = coef_table[coef_table['
    Importance'] > 0.01]['Feature'].tolist()
X_reduced = X[important_features]
X_train, X_test, y_train, y_test = train_test_split(
    X_reduced, y, test_size=0.2, random_state=42)
rf_model = RandomForestRegressor(n_estimators=100,
    random_state=42)
rf_model.fit(X_train, y_train)
y_pred = rf_model.predict(X_test)
r2 = r2_score(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
print(f"\nModel R: {r2:.3f}, MSE: {mse:.2f}")
coef_reduced = pd.DataFrame({
    "Feature": X_reduced.columns,
    "Importance": rf_model.feature_importances_
}).sort_values(by="Importance", ascending=False)

print("\nFeature Importance Table")
print(coef_reduced)
plt.figure(figsize=(8, 5))
plt.scatter(y_test, y_pred, alpha=0.5, color='blue')
plt.plot([y.min(), y.max()], [y.min(), y.max()],
    color='red', linestyle='--')
plt.xlabel("Actual MntWines")
plt.ylabel("Predicted MntWines")
plt.title("Predicted vs Actual Red Wine Spending (
    Reduced Features)")
plt.grid(True)
plt.tight_layout()
plt.show()
cv_scores = cross_val_score(rf_model, X_reduced, y,
    cv=5, scoring='r2')
print("Cross_Validation R score:", cv_scores)
print(f"mean R:{cv_scores.mean():.3f}")
import joblib
joblib.dump(rf_model, 'RFMODEL.pkl')
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV,
    train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X_reduced, y, test_size=0.2, random_state=42
)
param_grid = {
    'n_estimators': [200, 400, 600],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
rf = RandomForestRegressor(random_state=42)
grid = GridSearchCV(
    rf, param_grid,
    cv=5,
    scoring='r2',
    n_jobs=-1,
    verbose=1
)
grid.fit(X_train, y_train)
best_model = grid.best_estimator_
print("Best Parameter:", grid.best_params_)
print(f"Validator Set R: {grid.best_score_:.3f}")
y_pred = best_model.predict(X_test)
print(f"Test Set R: {r2_score(y_test, y_pred):.3f}")
print(f"Test Set MSE: {mean_squared_error(y_test,
    y_pred):.2f}")
import matplotlib.pyplot as plt
import seaborn as sns
importances = best_model.feature_importances_
feat_imp = pd.Series(importances, index=X_reduced.
    columns).sort_values(ascending=False)[:10]
plt.figure(figsize=(6,4))
sns.barplot(x=feat_imp.values, y=feat_imp.index,
```

```
    palette="Blues_d")
plt.xlabel("Feature Importance")
plt.title("Top-10 Feature Importances (Random Forest)
    ")
plt.tight_layout()
plt.show()
import numpy as np
residuals = y_test - y_pred
plt.figure(figsize=(6,4))
sns.histplot(residuals, bins=30, kde=True)
plt.axvline(0, color='red', linestyle='--')
plt.xlabel("Residual (Actual  Predicted)")
plt.title("Residual Distribution")
plt.tight_layout()
plt.show()
```

C. Appendix: Elena's Code

Listing 3. Elena's Python code

```
# Model selection
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score,
    precision_score, recall_score, make_scorer
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.feature_selection import RFECV
from sklearn.svm import LinearSVC
import warnings

warnings.filterwarnings('ignore')  # To suppress
    convergence warnings

# --- 1. Load & preprocess your data ---
df = pd.read_csv("marketing_campaign_CLEANED.csv",
    sep=',')
df.dropna(inplace=True)

df['Age'] = 2015 - df['Year_Birth']
df['TotalSpent'] = df[['MntWines', 'MntFruits', '
    MntMeatProducts',
                       'MntFishProducts', '
    MntSweetProducts', 'MntGoldProds']].sum(axis=1)
df['PurchaseType'] = df.apply(lambda row: 1 if row['
    NumStorePurchases'] > row['NumWebPurchases'] else
     0, axis=1)

features = ['Age', 'Income', 'Education', '
    Marital_Status', 'Kidhome','Teenhome',
           'TotalSpent','MntWines', 'MntFruits', '
    MntMeatProducts','MntFishProducts',
           'MntSweetProducts', 'MntGoldProds','
    Recency','NumDealsPurchases', '
    NumCatalogPurchases',
           'NumWebVisitsMonth','Complain','Response'
    ]
X = df[features]
y = df['PurchaseType']

# Encode categoricals for modeling
X_encoded = pd.get_dummies(X, drop_first=True)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, stratify=y, test_size=0.2,
    random_state=42)

# --- 2. Custom scoring function (balance of acc,
    precision, recall) ---

def balanced_gap_score(y_true, y_pred):

    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, pos_label
    =1, zero_division=0)
    rec = recall_score(y_true, y_pred, pos_label=1,
    zero_division=0)

    std = np.std([acc, prec, rec])
    avg = np.mean([acc, prec, rec])

    return avg - std

scorer = make_scorer(balanced_gap_score)

# --- 3. Define your models to test ---
models = {
    "Random Forest": RandomForestClassifier(
    n_estimators=100, random_state=42),
    "Logistic Regression": LogisticRegression(
    max_iter=10000, random_state=42),
    "Support Vector Machine": LinearSVC(random_state
    =42),
    "Decision Tree": DecisionTreeClassifier(
    random_state=42)
}

# --- 4. Loop through and evaluate models using RFECV
     ---
results = []

for name, model in models.items():
    print(f"Training {name}...")

    if model == 'Support Vector Machine':
        from sklearn.preprocessing import
    StandardScaler

        scaler = StandardScaler()
        X_train = scaler.fit_transform(X_train)
        rfe = RFE(estimator=SVC(kernel='linear'),
    n_features_to_select=10)  # RFE uses linear SVM
    internally
        rfe.fit(X_train, y_train)
        selected_features = rfe.transform(X_train)



    else:
        selector = RFECV(estimator=model, step=1, cv
    =5, scoring=scorer)
        selector.fit(X_train, y_train)

        selected_features = X_train.columns[selector.
    support_]

    # Train model on selected features
    model.fit(X_train[selected_features], y_train)
    y_pred = model.predict(X_test[selected_features])
```

```
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, pos_label
    =1, zero_division = 0)
    prec2 = precision_score(y_test, y_pred, pos_label
    =0, zero_division = 0)
    rec = recall_score(y_test, y_pred, pos_label=1,
    zero_division = 0)
    rec2 = recall_score(y_test, y_pred, pos_label=0,
    zero_division = 0)
    score = balanced_gap_score(y_test, y_pred)

    print(f"{name}:")
    print(f"␣␣Selected␣Features:␣{selected_features.
    tolist()}")
    print(f"␣␣Accuracy:␣{acc:.3f},␣Precision:␣{prec
    :.3f},␣Recall:␣{rec:.3f},␣Balanced␣Score:␣{score
    :.3f}\n")

    results.append({
        "Model": name,
        "Features": selected_features.tolist(),
        "Accuracy": acc,
        "Precision": prec,
        'Precision2':prec2,
        "Recall": rec,
        'Recall2':rec2,
        "Balanced␣Score": score
    })

# --- 5. Optional: Compare results ---
results_df = pd.DataFrame(results).sort_values(by="
    Balanced␣Score", ascending=False)
print("Summary␣of␣Results:")
print(results_df[['Model', 'Accuracy', 'Precision', '
    Recall', 'Balanced␣Score']])
```

```
# model training
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

from sklearn.model_selection import train_test_split,
     cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score,
    precision_score, recall_score,
    classification_report, confusion_matrix,
    ConfusionMatrixDisplay
from sklearn.feature_selection import RFECV


def balanced_gap_score(y_true, y_pred):
    from sklearn.metrics import accuracy_score,
    precision_score, recall_score
    import numpy as np

    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, pos_label
    =1, zero_division=0)
    rec = recall_score(y_true, y_pred, pos_label=1,
    zero_division=0)
    std = np.std([acc, prec, rec])
    avg = np.mean([acc, prec, rec])

    return avg - std

scorer = make_scorer(balanced_gap_score)

# Load the dataset
df = pd.read_csv("marketing_campaign_CLEANED.csv",
    sep=",")  # Adjust path/sep if needed

# Feature engineering (example: create 'Age')
df['Age'] = 2021 - df['Year_Birth']  # Dataset is
    from 2015
df['TotalSpent'] = df[['MntWines', 'MntFruits', '
    MntMeatProducts',
                       'MntFishProducts', '
    MntSweetProducts', 'MntGoldProds']].sum(axis=1)
# Define target variable
# Custom logic: classify whether the customer prefers
     in-store or online purchases
df['PurchaseType'] = df.apply(lambda row: 'store' if
    row['NumStorePurchases'] > row['NumWebPurchases']
     else 'online', axis=1)

# Encode target for heatmap
df['PurchaseTypeNum'] = df['PurchaseType'].map({'
    store': 0, 'online': 1})


# ---  Plot heatmap of feature correlations ---
corr_features = df.select_dtypes(include=['float64',
    'int64']).copy()
corr_features['PurchaseTypeNum'] = df['
    PurchaseTypeNum']
corr_matrix = corr_features.corr()

plt.figure(figsize=(10, 6))
sns.heatmap(corr_matrix[['PurchaseTypeNum']].
    sort_values(by='PurchaseTypeNum', ascending=False
    ),
            annot=True, cmap='coolwarm', linewidths
    =0.5)
plt.title('Feature␣Correlation␣with␣Target␣(Purchase␣
    Type)')
plt.tight_layout()
plt.show()


# Define features and target
features =  ['Age', 'Income', 'Education', '
    Marital_Status', 'Kidhome','Teenhome',
            'TotalSpent','MntWines', 'MntFruits', '
    MntMeatProducts','MntFishProducts',
            'MntSweetProducts', 'MntGoldProds','
    Recency','NumDealsPurchases', '
    NumCatalogPurchases',
            'NumWebVisitsMonth','Complain','Response'
    ]
X = df[features]  # Add more as needed
y = df['PurchaseTypeNum']

# Preprocessing for numerical and categorical columns
#numerical = ['Age', 'Income', 'TotalSpent', 'Kidhome
    ','NumCatalogPurchases','Recency',
             #'NumDealsPurchases','NumWebVisitsMonth
```

```
    ','Complain','Response','MntWines', 'MntFruits',
    'MntMeatProducts','MntFishProducts',
           #'MntSweetProducts', 'MntGoldProds']
#categorical = ['Education', 'Marital_Status']


# --- Preprocessing + RFECV ---
# Encode categoricals first for RFECV
X_encoded = pd.get_dummies(X, drop_first=True)

# Train/test/val split
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, stratify=y, test_size=0.2,
    random_state=1)

X_test, X_val, y_test, y_val = train_test_split(
    X_test, y_test, test_size=0.5, random_state=1) #
    0.5 x 0.2 = 0.1

# Basic preprocessing
# Perform Median Imputation on 'Income'
# Calculate the median ONLY from the training set to
    prevent data leakage.
median_income = X_train['Income'].median()

# Impute missing values in training, validation, and
    test sets using this calculated median
X_train['Income'] = X_train['Income'].fillna(
    median_income)
X_val['Income'] = X_val['Income'].fillna(
    median_income)
X_test['Income'] = X_test['Income'].fillna(
    median_income)

print("\nMissing values in Income after imputation:")
print(f"X_train Income missing: {X_train['Income'].
    isnull().sum()}")
print(f"X_val Income missing:    {X_val['Income'].
    isnull().sum()}")
print(f"X_test Income missing:   {X_test['Income'].
    isnull().sum()}")

# RFECV with Random Forest
rf = RandomForestClassifier(n_estimators=100,
    random_state=0)
selector = RFECV(estimator=rf, step=1, cv=5, scoring=
    scorer)
selector.fit(X_train, y_train)

# Output selected features
selected_features = X_encoded.columns[selector.
    support_]
print("Selected features after RFECV:",
    selected_features.tolist())

# Train final model on selected features
rf_final = RandomForestClassifier(n_estimators=17,
    random_state=0)
rf_final.fit(X_train[selected_features], y_train)
y_train_pred = rf_final.predict(X_train[
    selected_features])
print("Training model accuracy:", accuracy_score(
    y_train, y_train_pred))
print("Training model precision:", precision_score(
    y_train, y_train_pred))
print("Training model recall:", recall_score(y_train,
     y_train_pred))
cm = confusion_matrix(y_train, y_train_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
    display_labels=rf_final.classes_)
disp.plot()
plt.show()

y_pred = rf_final.predict(X_val[selected_features])
print("Final model accuracy:", accuracy_score(y_val,
    y_pred))
print("Final model precision:", precision_score(y_val
    , y_pred))
print("Final model recall:", recall_score(y_val,
    y_pred))
print('Final model balanced score:',
    balanced_gap_score(y_val,y_pred))
#print(classification_report(y_val, y_pred))
cm = confusion_matrix(y_val, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
    display_labels=rf_final.classes_)
disp.plot()
plt.show()

# --- Plot accuracy vs n_estimators ---
train_scores = []
val_scores = []
prec_scores = []
rec_scores = []
estimators_range = list(range(1, 101, 2))

for n in estimators_range:
    rf = RandomForestClassifier(n_estimators=n,
    random_state=0)
    rf.fit(X_train[selected_features], y_train)
    #ypred = rf.predict(X_test)
    train_scores.append(accuracy_score(y_train, rf.
    predict(X_train[selected_features])))
    val_scores.append(accuracy_score(y_val, rf.
    predict(X_val[selected_features])))
    prec_scores.append(precision_score(y_val, rf.
    predict(X_val[selected_features])))
    rec_scores.append(recall_score(y_val, rf.predict(
    X_val[selected_features])))


#plt.figure(figsize=(10, 6))
plt.figure()
plt.plot(estimators_range, train_scores, label="
    Training Accuracy", marker='o')
plt.plot(estimators_range, val_scores, label="
    Validation Accuracy", marker='o')
plt.plot(estimators_range, prec_scores, label="
    Validation Precision", marker='o')
plt.plot(estimators_range, rec_scores, label="
    Validation Recall", marker='o')
plt.xlabel("Number of Trees (n_estimators)")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

rf_final.fit(X_train[selected_features], y_train)
y_test_pred = rf_final.predict(X_test[
    selected_features])
print("Tested model accuracy:", accuracy_score(y_test
```

```
    , y_test_pred))
print("Tested model precision:", precision_score(
    y_test, y_test_pred))
print("Tested model recall:", recall_score(y_test,
    y_test_pred))
#print('Final model balanced score:', balanced_score(
    y_test,y_test_pred))
#print(classification_report(y_val, y_pred))
cm = confusion_matrix(y_test, y_test_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
    display_labels=rf_final.classes_)
disp.plot()
plt.show()
```

D. Appendix: Leia's Code

Listing 4. Leia's Python code

```
# -*- coding: utf-8 -*-
"""Untitled6.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1
    fZA6_AqBlsNA8oBQM8dg-aeFB7wcFvsW
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
from sklearn.model_selection import train_test_split,
     KFold, GridSearchCV
from sklearn.linear_model import LinearRegression,
    Lasso # Import Lasso
from sklearn.metrics import mean_absolute_error,
    mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE # Not used
    for this regression problem
import kagglehub
from kagglehub import KaggleDatasetAdapter

# Initial Cleaning

#importing dataset
file_path = "marketing_campaign.csv"
df_original = kagglehub.load_dataset(
    KaggleDatasetAdapter.PANDAS,
    "imakash3011/customer-personality-analysis",
    file_path,
    pandas_kwargs={'sep': '\t'}
)


#making age feature
current_year_for_age_calc = 2014
df_original['Age'] = current_year_for_age_calc -
    df_original['Year_Birth']

df = df_original[df_original['Age'] < 120].copy()
print({len(df_original)})
print({len(df)})


#filling missing income values
median_raw_income = df['Income'].median()
df['Income'] = df['Income'].fillna(median_raw_income)


import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np



#Making Income distribution histogram

plt.figure(figsize=(8, 5))

sns.histplot(
    df['Income'],
    kde=True,
    bins=50,
    color='skyblue',
    edgecolor='black',
    linewidth=0.5   )

# Add title and labels
plt.title('Distribution of Annual Income', fontsize
    =16)
plt.xlabel('Annual Income ()', fontsize=14) # Use
    appropriate units if known
plt.ylabel('Frequency / Density', fontsize=14)

# Add gridlines
plt.grid(axis='y', alpha=0.75, linestyle='--')
plt.grid(axis='x', alpha=0.5, linestyle=':')

plt.ticklabel_format(style='plain', axis='x')

# Add mean and median lines
plt.axvline(df['Income'].mean(), color='red',
    linestyle='dashed', linewidth=1.5, label=f'Mean:
    {df["Income"].mean():,.0f}')
plt.axvline(df['Income'].median(), color='green',
    linestyle='dashed', linewidth=1.5, label=f'Median
    :{df["Income"].median():,.0f}')
plt.legend(fontsize=10)

plt.tight_layout()
plt.show()



#log transform income
df['Income_Log1p'] = np.log1p(df['Income'])


#remove 'YOLO' and 'Absurd'
unusual_marital_statuses = ['YOLO', 'Absurd']
initial_rows_before_marital_status_clean = len(df)
df = df[~df['Marital_Status'].isin(
    unusual_marital_statuses)].copy()
rows_removed_marital_status =
    initial_rows_before_marital_status_clean - len(df
    )
```

```
# Data Splitting

y = df['Income_Log1p']
X = df.drop(['Income', 'Income_Log1p', 'Complain', '
    ID', 'Year_Birth', 'Z_CostContact', 'Z_Revenue'],
     axis=1)

X_train, X_temp, y_train, y_temp = train_test_split(X
    , y, test_size=0.3, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42)



#Numerical Feature Preprocessing


# Making box plots
numerical_cols_for_viz = [
    'Age', 'Recency', 'MntWines', 'MntFruits', '
    MntMeatProducts',
    'MntFishProducts', 'MntSweetProducts', '
    MntGoldProds',
    'NumDealsPurchases', 'NumWebPurchases', '
    NumCatalogPurchases',
    'NumStorePurchases', 'NumWebVisitsMonth',
    'Kidhome', 'Teenhome'
]

plt.figure(figsize=(18, 12))
for i, col in enumerate(numerical_cols_for_viz):
    if col in X_train.columns:
        plt.subplot(4, 4, i + 1) # Max 16 plots for 4
    x4
        sns.boxplot(y=X_train[col])
        plt.title(f'Box␣plot␣of␣{col}')
        plt.ylabel('')
plt.tight_layout()
plt.show()

# Implement Outlier Treatment

log_transform_cols = [
    'MntWines', 'MntFruits', 'MntMeatProducts',
    'MntFishProducts', 'MntSweetProducts', '
    MntGoldProds'
]
for col in log_transform_cols:
    if col in X_train.columns:
        X_train[col] = np.log1p(X_train[col])
        X_val[col] = np.log1p(X_val[col])
        X_test[col] = np.log1p(X_test[col])


cols_for_capping = [
    'Recency', 'NumDealsPurchases', 'NumWebPurchases'
    ,
    'NumCatalogPurchases', 'NumStorePurchases', '
    NumWebVisitsMonth', 'Age',
    'Kidhome', 'Teenhome'
]
for col in cols_for_capping:
    if col in X_train.columns:
        Q1 = X_train[col].quantile(0.25)
        Q3 = X_train[col].quantile(0.75)
        IQR = Q3 - Q1
        upper_bound = Q3 + 1.5 * IQR
        lower_bound = Q1 - 1.5 * IQR
        X_train[col] = np.clip(X_train[col],
    lower_bound, upper_bound)
        X_val[col] = np.clip(X_val[col], lower_bound,
     upper_bound)
        X_test[col] = np.clip(X_test[col],
    lower_bound, upper_bound)


#Dt_Customer handing

date_format = '%d-%m-%Y'
for df_split in [X_train, X_val, X_test]:
    if 'Dt_Customer' in df_split.columns:
        df_split['Dt_Customer'] = pd.to_datetime(
    df_split['Dt_Customer'], format=date_format)
if 'Dt_Customer' in X_train.columns:
    reference_date_for_tenure = X_train['Dt_Customer'
    ].max()
    print(f"\nReference␣date␣for␣tenure␣calculation␣(
    from␣X_train␣max␣Dt_Customer):␣{
    reference_date_for_tenure.strftime('%Y-%m-%d')}")
    for df_split in [X_train, X_val, X_test]:
        df_split['Customer_Tenure_Days'] = (
    reference_date_for_tenure - df_split['Dt_Customer
    ']).dt.days
        if df_split is X_train: df_split.name = '
    X_train'
        elif df_split is X_val: df_split.name = '
    X_val'
        elif df_split is X_test: df_split.name = '
    X_test'
    for df_split in [X_train, X_val, X_test]:
        df_split.drop('Dt_Customer', axis=1, inplace=
    True)


#Feature Engineering


mnt_cols_to_sum_for_total = [
    'MntWines', 'MntFruits', 'MntMeatProducts', '
    MntFishProducts',
    'MntSweetProducts', 'MntGoldProds'
]
for df_split in [X_train, X_val, X_test]:
    df_split['Total_Mnt'] = df_split[
    mnt_cols_to_sum_for_total].sum(axis=1)
num_purch_cols = [
    'NumDealsPurchases', 'NumWebPurchases', '
    NumCatalogPurchases',
    'NumStorePurchases'
]
for df_split in [X_train, X_val, X_test]:
    df_split['Total_Purchases'] = df_split[
    num_purch_cols].sum(axis=1)

for df_split in [X_train, X_val, X_test]:
    df_split['Kids'] = df_split['Kidhome'] + df_split
    ['Teenhome']
age_bins = [0, 25, 40, 60, 120]
age_labels = ['Young', 'Adult', 'Middle-Aged', '
    Senior']
if 'Age' in X_train.columns:
    for df_split in [X_train, X_val, X_test]:
        df_split['Age_Group'] = pd.cut(df_split['Age'
```

```
    ], bins=age_bins, labels=age_labels, right=False,
     include_lowest=True)


# Categorical Variable Encoding

categorical_cols = X_train.select_dtypes(include=['
    object', 'category']).columns.tolist()
if 'Dt_Customer' in categorical_cols:
    categorical_cols.remove('Dt_Customer')
train_size = X_train.shape[0]
val_size = X_val.shape[0]
X_combined = pd.concat([X_train, X_val, X_test],
    ignore_index=True)
X_combined_encoded = pd.get_dummies(X_combined,
    columns=categorical_cols, drop_first=True, dtype=
    int)
X_train_encoded = X_combined_encoded.iloc[:train_size
    , :].copy()
X_val_encoded = X_combined_encoded.iloc[train_size :
    train_size + val_size, :].copy()
X_test_encoded = X_combined_encoded.iloc[train_size +
     val_size :, :].copy()
X_train = X_train_encoded
X_val = X_val_encoded
X_test = X_test_encoded

print("Categorical␣variable␣encoding␣complete.")

# Numerical Feature Scaling

numerical_cols_for_scaling = [
    'Age', 'Recency', 'Customer_Tenure_Days',
    'MntWines', 'MntFruits', 'MntMeatProducts', '
    MntFishProducts',
    'MntSweetProducts', 'MntGoldProds',
    'NumDealsPurchases', 'NumWebPurchases', '
    NumCatalogPurchases',
    'NumStorePurchases', 'NumWebVisitsMonth',
    'Kidhome', 'Teenhome', 'Kids',
    'Total_Mnt', 'Total_Purchases',
    'AcceptedCmp1', 'AcceptedCmp2', 'AcceptedCmp3', '
    AcceptedCmp4', 'AcceptedCmp5', 'Response' #
    Binary features. Scale if desired, often
    beneficial for LR.
]
# Ensure only columns present in X_train are selected
final_numerical_features_to_scale = [col for col in
    numerical_cols_for_scaling if col in X_train.
    columns]

scaler = StandardScaler()
scaler.fit(X_train[final_numerical_features_to_scale
    ])
X_train[final_numerical_features_to_scale] = scaler.
    transform(X_train[
    final_numerical_features_to_scale])
X_val[final_numerical_features_to_scale] = scaler.
    transform(X_val[final_numerical_features_to_scale
    ])
X_test[final_numerical_features_to_scale] = scaler.
    transform(X_test[
    final_numerical_features_to_scale])
print("\nStandardScaler␣fitted␣and␣transformed␣
    numerical␣columns␣across␣splits.")
print(f"\nFirst␣5␣rows␣of␣X_train␣after␣scaling:\n{
    X_train.head()}")
print("\n---␣Numerical␣Feature␣Scaling␣complete.␣---"
    )

# Model applied with LASSO

base_lasso_model = Lasso(random_state=42, max_iter
    =2000)
param_grid_lasso = {
    'alpha': [0.0001, 0.001, 0.01, 0.1, 1.0, 10.0]
}
scoring_metric_lasso = 'r2'
cv_strategy_lasso = KFold(n_splits=5, shuffle=True,
    random_state=42)

grid_search_lasso = GridSearchCV(
    estimator=base_lasso_model,
    param_grid=param_grid_lasso,
    scoring=scoring_metric_lasso,
    cv=cv_strategy_lasso,
    n_jobs=-1,
    verbose=1,
    return_train_score=True
)

grid_search_lasso.fit(X_train, y_train)

best_lasso_model = grid_search_lasso.best_estimator_

lasso_selected_features_mask = (best_lasso_model.
    coef_ != 0)
lasso_selected_features_names = X_train.columns[
    lasso_selected_features_mask].tolist()

if not lasso_selected_features_names:
    lasso_selected_features_names = list(X_train.
    columns)

results_df_lasso = pd.DataFrame(grid_search_lasso.
    cv_results_)
plt.figure(figsize=(10, 6))
plt.plot(results_df_lasso['param_alpha'],
    results_df_lasso['mean_test_score'], marker='o',
    label='Mean␣Test␣R2', color='blue')
plt.plot(results_df_lasso['param_alpha'],
    results_df_lasso['mean_train_score'], marker='x',
     label='Mean␣Train␣R2', color='orange')
plt.xscale('log')
plt.title('R-squared␣vs.␣Alpha␣(LASSO␣Regression␣
    Cross-Validation)')
plt.xlabel('Alpha␣(Regularization␣Strength)')
plt.ylabel('R-squared')
plt.grid(True)
plt.legend()
plt.show()

model_final = LinearRegression()
model_final.fit(X_train[lasso_selected_features_names
    ], y_train)

y_val_pred_final = model_final.predict(X_val[
    lasso_selected_features_names])

mae_final_val = mean_absolute_error(y_val,
    y_val_pred_final)
mse_final_val = mean_squared_error(y_val,
    y_val_pred_final)
rmse_final_val = np.sqrt(mse_final_val)
```

```
r2_final_val = r2_score(y_val, y_val_pred_final)

print(f"MAE (Final Model validation):     {
    mae_final_val:.4f}")
print(f"MSE (Final Model validation):     {
    mse_final_val:.4f}")
print(f"RMSE (Final Model validation):    {
    rmse_final_val:.4f}")
print(f"R-squared (R2) (Final Model validation): {
    r2_final_val:.4f}")

feature_names_final = lasso_selected_features_names
coefficients_final = model_final.coef_

feature_importance_df_final = pd.DataFrame({
    'Feature': feature_names_final,
    'Coefficient': coefficients_final
})
feature_importance_df_final['Abs_Coefficient'] = abs(
    feature_importance_df_final['Coefficient'])
feature_importance_df_final =
    feature_importance_df_final.sort_values(by='
    Abs_Coefficient', ascending=False)

print(feature_importance_df_final)

y_test_pred_final = model_final.predict(X_test[
    lasso_selected_features_names])

mae_final_test = mean_absolute_error(y_test,
    y_test_pred_final)
mse_final_test = mean_squared_error(y_test,
    y_test_pred_final)
rmse_final_test = np.sqrt(mse_final_test)
r2_final_test = r2_score(y_test, y_test_pred_final)

print(f"MAE on Test Set (Final Model):     {
    mae_final_test:.4f}")
print(f"MSE on Test Set (Final Model):     {
    mse_final_test:.4f}")
print(f"RMSE on Test Set:    {rmse_final_test:.4f}")
print(f"R-squared (R2) on Test Set (Final Model): {
    r2_final_test:.4f}")
```